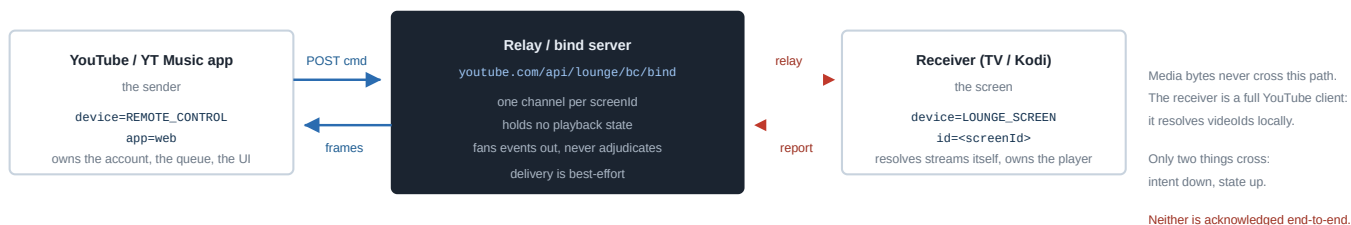


State transfer in YouTube Lounge casting

How the official protocol moves state, how the Kodi receiver moves it, and why the two drift apart

Casting a YouTube video to a TV looks like one action. Underneath it is two independent clients projecting state at each other through a relay they do not control, with no acknowledgement layer, no authoritative state store, and no way to ask “what is actually true right now?”. Every sync bug in a third-party receiver follows from that architecture — not from any single missing report.



One endpoint, one channel, two directions: this is the whole architecture.

Fig. 0 The three roles of a cast session. The cloud is a message relay with an identity table — not a source of truth.

WHAT THIS DOCUMENT DOES

Reconstructs the state-transfer model of the official Lounge implementation (§2–§5), dissects the Kodi receiver's implementation of the same protocol (§6), measures what the wire actually carried on a real device (§7), names the five structural causes behind the recurring sync bugs (§8), and specifies a smaller architecture that removes three of them outright (§9–§10).

EVIDENCE BASE

Everything marked **verified** is grounded in the receiver's own code, its emulator suite, the reverse-engineered protocol notes, or the 2026-09-14 device capture (512 receiver posts re-measured for this document). Nothing here is an inference about Google's internals stated as fact.

IN THIS DOCUMENT

§1 One endpoint, two RPC streams

§2 Pairing: making a screen addressable

§3 One cast, end to end

§4 How the official implementation moves state

§5 What actually crosses the wire

§6 How the Kodi add-on does it

§7 Measured reality: 512 posts

§8 Five structural causes of sync bugs

§9 The smaller architecture & publish contract

§10 Migration without a rewrite

App. Protocol reference & evidence index

12 figures · 13 pages

The protocol is not a state store — it is two RPC streams wearing one endpoint

Lounge (the “Watch on TV” / casting path for YouTube and YouTube Music) is documented nowhere by Google. What follows is what the clients demonstrably do. There are exactly three roles and exactly one transport:

Model, in one sentence: every participant — phone or screen — is a client of a *bidirectional channel* identified by a `screenId`; each one posts named RPC calls into that channel and reads everyone else's RPC calls back out as framed events. The server routes; it does not remember playback.

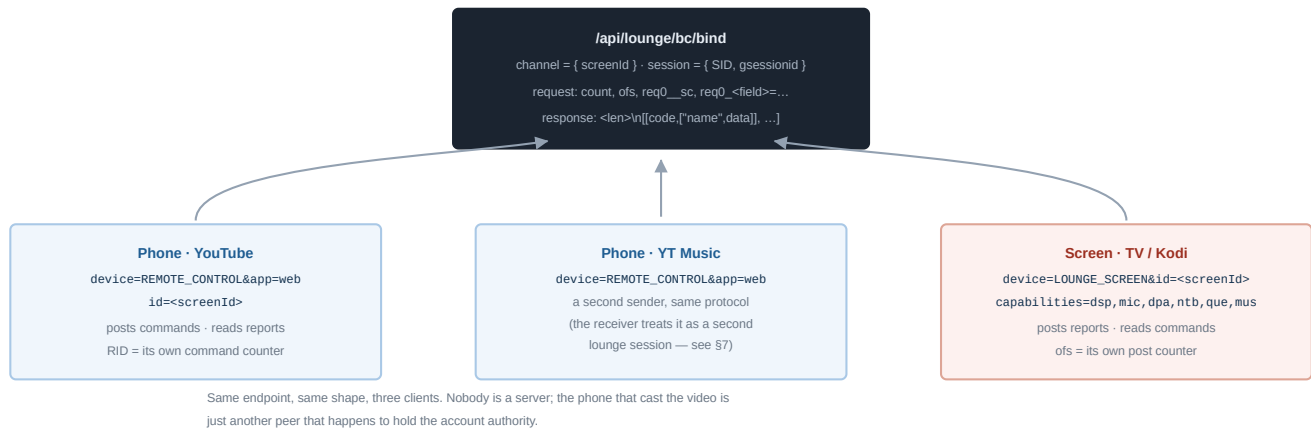


Fig. 1 The channel topology. verified from receiver code, sender reference implementations and the device capture.

1.1 The five mechanisms that carry all state

MECHANISM	WHAT IT DOES	SERVES AS AN ACK?
Channel identity <code>screenId</code>	The single key that binds every participant. A sender attaches with <code>id=<screenId></code> ; the screen binds as <code>LOUNGE_SCREEN</code> with the same id.	No — identity only
Session pair <code>SID</code> , <code>gsessionId</code>	Issued once in the handshake reply (<code>count=0</code> → frames <code>c</code> and <code>S</code>). Required on every later poll, or the stream is not the same session.	No — routing only
Frame code	Per-channel monotonic index on each inbound frame. The receiver's only defence against replaying a command twice is <code>if code > last_code</code> .	Dedupe input, not delivery
Post ofs	Monotonic counter on each outbound RPC batch. The server uses it to order/dedupe posts from <i>that one client</i> . Duplicate or colliding <code>ofs</code> means the report is dropped.	Ordering, not delivery
RPC name <code>req0__sc=<name></code>	Each direction names its calls. Inbound to the screen: <code>setPlaylist</code> , <code>play</code> , <code>pause</code> , <code>seekTo</code> , <code>setVolume</code> , <code>updatePlaylist</code> , <code>getNowPlaying</code> . Outbound: <code>nowPlaying</code> , <code>onStateChange</code> , <code>onVolumeChanged</code> , <code>nowPlayingPlaylist</code> , ...	No — a report is a statement, not a response to the command it answers

The consequence that matters: nothing in this list confirms that a *receiver report* reached the phone, and nothing confirms that a *phone command* reached the screen. A cast session is two independently published narratives that the humans on both ends interpret as one shared truth.

Before any state: how a screen becomes addressable

Pairing exists to solve one problem — the phone must be able to name a destination. Google solved it with a phone-shows-you-a-code dance and a durable identity pair: `screenId` (the channel key, publicly reachable) and `loungeIdToken` (the screen's credential on that channel). The receiver never authenticates an account; it only registers a device identity. All account authority stays in the phone.

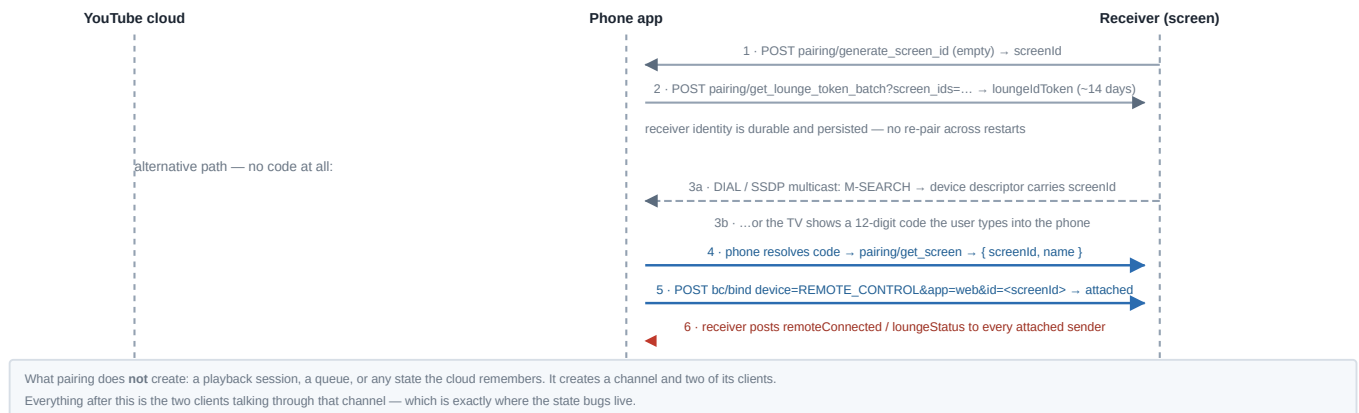


Fig. 2 Pairing is identity exchange, not session creation. verified against the receiver's pairing module and the reverse-engineered sender.

2.1 The two identity artefacts

ARTEFACT	LIFETIME	HELD BY	WHAT BREAKS WITHOUT IT
<code>screenId</code>	stable while the receiver lives	both sides — it is the channel key	Nothing routes: the phone's target and the screen's poll are different channels, so casting hangs after discovery. Persisting it is what makes pairing survive a restart.
<code>loungeIdToken</code>	~14 days, refreshed	the receiver only	The bind is rejected (HTTP 400/404) and the screen must re-register and show a new code — while the phone still lists the old device, which is why "it disappears from the cast list at random" is usually a token expiry, not a network fault.
<code>deviceId</code> + <code>screenName</code>	persistent	the receiver only	A regenerated <code>deviceId</code> on every boot looks like a new TV to the user and breaks "permanent" pairing semantics — the receiver must reuse it across restarts.

The rule this implies for the receiver: pairing state is durable state. Anything that makes a screen addressable must survive a process restart, or the phone and the screen will silently disagree about which device the user is casting to. The same rule, applied one level up, is what §9 R2 asks for the *playback* session — the failure mode is identical, only the symptom is quieter.

One cast, end to end: who tells whom what, and when



Fig. 3 The full life of a cast. Everything the phone renders comes from the red arrows; everything the screen does comes from the blue ones. The three failure classes at the bottom are analysed in §8.

3.1 The command surface, and what each command obliges the receiver to do

/ pause

COMMAND	CARRIES	RECEIVER OBLIGATION
setPlaylist	queue (videoIds), item, currentIndex , currentTime , queue key (listId)	Adopt the whole queue as the new session identity, resolve and start the item at the given offset, and remember listId for every subsequent report.
updatePlaylist	a revised queue	Replace the stored queue, re-derive currentIndex against it, continue playing.
play	nothing	Idempotent state change — and the receiver must not echo a conflicting state back, or the phone answers with a correction and the two flap.
seekTo	newTime	Seek, then report a position consistent with the seek. A seek arriving before playback has started must be honoured as a start offset, not dropped.
setVolume	volume	Apply and report back via onVolumeChanged .
getNowPlaying	nothing	Answer with a fresh snapshot — the phone's explicit way of asking “what is true?”, and the only recovery hook the protocol offers.
next / previous	nothing	Move within the stored queue and report the new item with the same listId and a re-derived index.

How the official implementation transfers state

Strip the protocol down and the official design is a deliberate asymmetry: **the phone is the authority on intent, the screen is the authority on reality, and neither is authoritative about the whole**. There is no message that says “here is the complete state”. There are named deltas, published by whoever happens to know them.

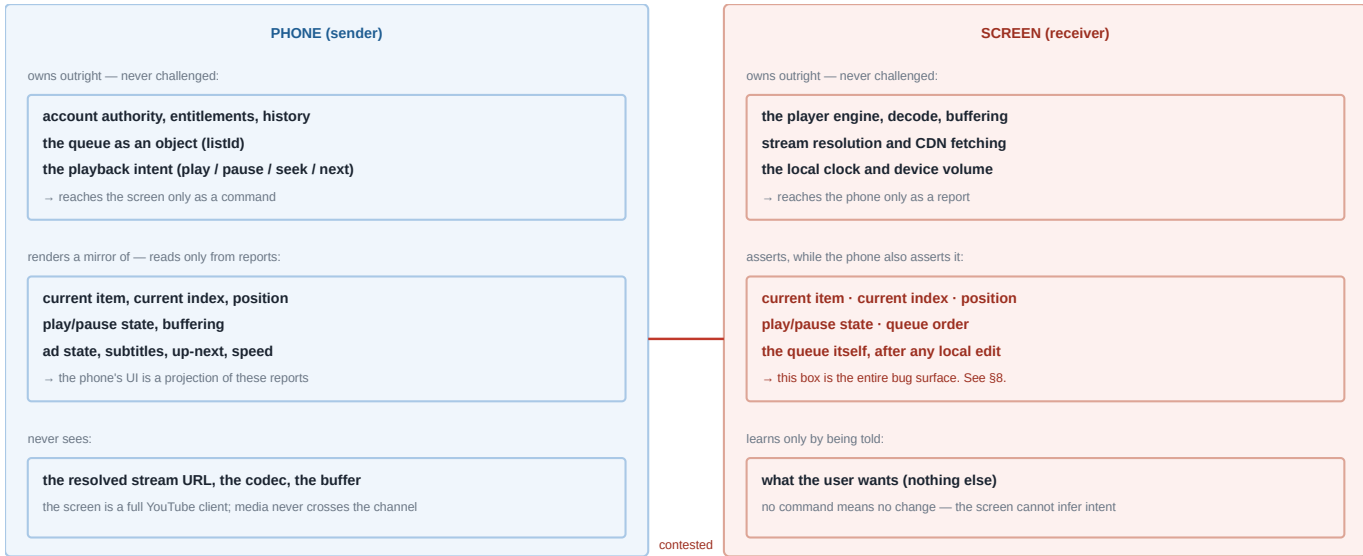


Fig. 4 Where each class of fact lives. State-sync bugs are, without exception, disagreements inside the two red boxes.

4.1 The two directions, formally

DIRECTION	MESSAGES	PAYLOAD CHARACTER	TIME MODEL
Down phone → screen	<code>setPlaylist</code> , <code>updatePlaylist</code> , <code>play</code> , <code>pause</code> , <code>stopVideo</code> , <code>seekTo</code> , <code>setVolume</code> , <code>next / previous</code> , <code>skipAd</code> , <code>getNowPlaying</code>	Intent + identity. <code>setPlaylist</code> carries the whole queue (<code>videoIds</code>), the item (<code>videoId</code>), the position in it (<code>currentIndex</code>), the offset (<code>currentTime</code>) and the queue key (<code>listId</code>). Later commands are usually identity-free deltas.	Edge-triggered. Something is sent only when the user does something — so a lost message is permanently lost.
Up screen → phone	<code>nowPlaying</code> , <code>nowPlayingPlaylist</code> , <code>onStateChange</code> , <code>onVolumeChanged</code> , <code>onAdStateChange</code> , <code>onAdPlaying</code> , <code>onSubtitlesTrackChanged</code> , <code>autoplayUpNext</code> , <code>onPlaybackSpeedChanged</code> , <code>autoplayModeChanged</code>	Absolute snapshots. Each report is a self-contained picture of a few fields — there are no deltas and no sequence numbers beyond the sender's <code>ofs</code> .	Level-triggered. Reports repeat, which is what makes them survivable: a missing one is superseded by the next. Correctness depends on the reports remaining <i>self-consistent with each other</i> .

Why the vocabulary is as wide as it is

The phone's casting UI is not a remote control with a progress bar. It renders ad state, a countdown to skip, subtitle track selection, playback speed and an “up next” item — all of which only the screen can know. Each of those controls is backed by an event family. A receiver that implements four of the ten families does not merely have fewer features: the phone's model silently keeps defaults for the six it never hears about, which surfaces as controls that look present but do nothing.

4.2 What the asymmetry costs

Every fact both sides can observe is therefore asserted twice, and nothing decides which assertion wins — queue order, current item, position, play state and volume are all contested. §5 names the fields; §8 lists what happens when the two assertions diverge.

What actually crosses the wire

A · inbound event frame (phone → screen), exactly as captured

```
23\r\n32\r\n[[{"getDiscoveryDeviceId",...}]]
```

chunk size marker from the chunked HTTP body (read raw as 23\r\n)

frame length in bytes; the frame ends at exactly this boundary — parsers must be incremental

per-channel monotonic code → the receiver's only duplicate defence: if code > last_code

Handshake reply instead carries the session pair: [[{"c", "83802FC134080A01", "", 8}], [{"S", "kZLqQ1wzOM8U_wVZJnh2bfna5xAwmXMJ"}]]

B · outbound state report (screen → phone), one RPC batch

```
count=1&ofs=11&req0__sc=nowPlayingPlaylist&req0_videoIds=rILD9c2F9ZQ%2CNrZhrs3Buj4%2C...(27 ids)
&req0_videoId=rILD9c2F9ZQ&req0_currentIndex=1&req0_currentTime=0&req0_duration=205&req0_state=1
&req0_seekableStartTime=0&req0_seekableEndTime=205&req0_listId=RQyQVASPUitjBiVxn3y1ViezZvpk
```

- **ofs** — the batch offset. Colliding values are silently dropped server-side; unreadable ordering across channels (Fig. 7b).
- **req0__sc** — the RPC name decides everything downstream. Unknown names are ignored, not rejected.
- **videoids + videoId + currentIndex** — the queue as a flat list, the item, and its position. **This is identity.**
- **seekableStart/EndTime, loadedTime** — without them the phone calls the stream unseekable, zero-length or ended: spinner, restart button, dead scrubber.

Every one of these fields is mandatory-in-practice and none is validated by the relay: a malformed report returns HTTP 200.

Fig. 5 Frame and report anatomy from the 2026-09-14 device capture. Note what is *absent*: no request id tying a report to a command, no version, no ack, no error channel.

5.1 The three fields that make a report interpretable

FIELD	MEANING	FAILURE MODE WHEN WRONG
<code>listId</code>	The queue identity. Every report about a queue item must echo the <code>listId</code> the phone sent in <code>setPlaylist</code> .	A report carrying a new <code>videoId</code> under an unknown or missing <code>listId</code> cannot be bound to a slot in the phone's queue model, so the phone drops it and keeps rendering the previous item — bar at 0:00, restart glyph. Field symptom: <i>the TV plays the next track, the phone still shows the old one.</i>
<code>currentIndex</code>	Position of the current item <i>within that same queue</i> .	An index computed against a different list than the phone's is a second identity mismatch even when <code>listId</code> is correct, so every receiver-side queue edit invalidates it and requires re-derivation against the <i>stored</i> list.
<code>cpn</code>	Client playback nonce — identifies one playback instance, so the phone can tell a new session from a stale report about the old one.	A constant or absent <code>cpn</code> erases session boundaries: the phone cannot distinguish “same video restarted” from “previous video still playing”.

5.2 Transport, precisely — and what it hides

Transport, precisely

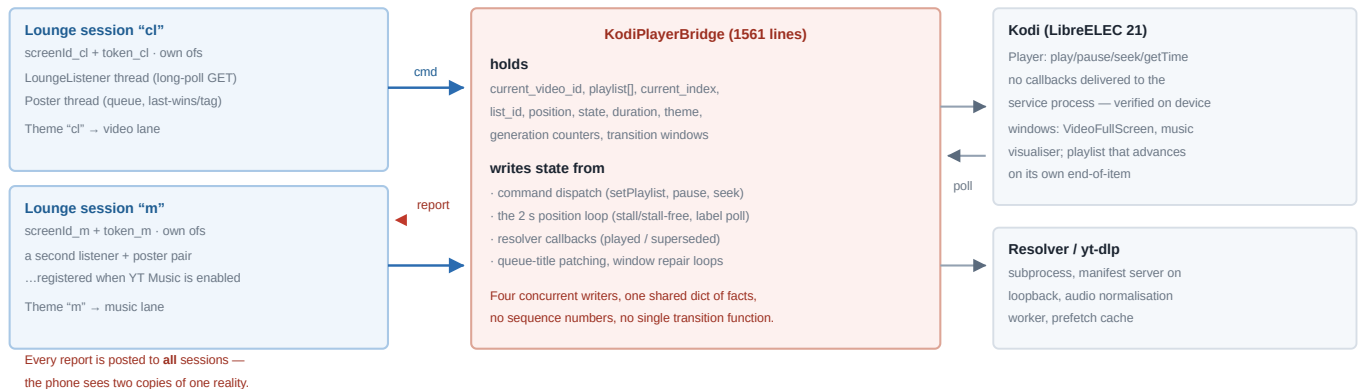
- **HTTP, not websockets, not protobuf.** Inbound events arrive as length-delimited JSON frames on a long-lived chunked `GET /bc/bind`; outbound reports are ordinary form-encoded `POST /bc/bind` bodies. Protobuf belongs to Cast V2 on port 8009 — a different protocol entirely, and a classic source of wrong designs.
- **Keepalive is idle traffic.** `noop` frames arrive roughly every 30 s on an open poll; a session that looks silent is not necessarily dead.
- **Session start broadcasts.** When a sender attaches, the relay pushes `getDiscoveryDeviceId` to every open poll — a burst of frames unrelated to playback.

The silent-failure property

Both directions return **HTTP 200 for anything well-formed**. An ignored field, a wrong `listId`, a report the phone cannot map, a command the screen never received — all of them look like success at the transport layer. There is no error path to build on, which is why **the only reliable debugging tool for this protocol is the receiver's own report log compared against what the phone actually renders.**

How the Kodi add-on does it

The add-on implements the receiver role honestly: no login, no protobuf, stdlib-only HTTP, the phone supplies all account authority, and videoIds are resolved locally with yt-dlp. The protocol layer is small and correct. The state layer is where the design diverges — and it diverges because **Kodi cannot be observed, only interrogated**.



Because events are unavailable, every fact is inferred by polling — and inference needs heuristics:

- `_position_loop()` wakes every 2 s and, while playing, publishes exactly three reports per session: `nowPlaying`, `nowPlayingPlaylist`, `onStateChange`.
 - **Pause** is inferred: two consecutive polls with a non-advancing clock, with hysteresis against demuxer hiccups. **Resume** only counts after ≥2 s of real progress — a short advance right after a pause is discarded, because treating it as a resume flipped the bridge to PLAYING and the phone answered with pause spam.
 - **Track change** is inferred from the info label `Player.FileNameAndPath` still carrying `?play=<id>`, because a Kodi-native auto-advance never tells the service anything.
 - **Lane and window** state (video window vs music visualiser) is repaired on every tick by `_repair_fullscreen_windows()` / `_ensure_music_window()`, and queue titles are patched after being snapshotted into the playlist.
- Each heuristic is a guess about physical reality re-run twice per second, forever. A guess that is wrong once produces a state report that is wrong until the next guess corrects it.

Fig. 6 The add-on's moving parts. The protocol layer is thin; the state layer absorbs Kodi's opacity, and that absorption is spread across four writers and a handful of repair loops.

What the add-on gets right

- Identity is durable: `screenId` + token persisted, no re-pair across restarts.
- Incremental frame parser that returns bytes consumed, so a partial trailing frame is never lost and never misparsed as a different command.
- `ofs` mutations are lock-protected, so the listener's handshake and the poster cannot collide offsets.
- Reports are posted off the listener thread (a queue + single poster), so a slow TLS post cannot stall command delivery.
- The emulator suite replays the real wire format and asserts on the reports — the correct testing model for this protocol.

What the add-on inherits and worsens

- **Fan-out by session, not by model.** Two channels, two offsets, two queues, every report written twice — 512 posts measured in one session (§7).
- **Coalescing with a hand-carved exception:** last-wins per tag, except `nowPlaying`, which is exempt because dropping it loses the "a new song started" signal.
- **Identity kept only in RAM:** `listId` lives in the bridge, so a restart mid-queue loses the key the phone's queue model is indexed by.
- **A constant `cpn` ("kodi")** in every report — playback sessions are indistinguishable to the phone.
- **Reports gated on `PLAYING`**: paused and stopped states are only emitted by transition handlers, so a missed transition leaves the phone with a stale play/pause glyph.

What the wire actually carried: a real session

The 2026-09-14 device capture records one evening of ordinary casting (`tv_full.jsonl`, 529 records, 512 outbound RPC batches). Re-measured for this document, it is a precise picture of the add-on's state-transfer behaviour in the field.

Fig. 1a Every state fact is transmitted in triplicate, every ~2 s, for as long as playback runs



Fig. 1b ...and the `ofs` offsets come from two counters, not one: two independent screens/sessions

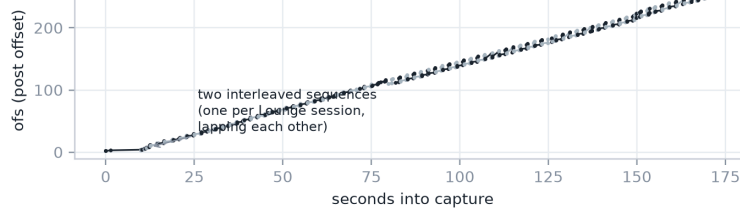


Fig. 7 Reports per second (top) and the post offsets they carried (bottom). The band structure is the 2 s tick; the doubling in the offset panel is two sessions posting the same facts.

MEASUREMENT	VALUE	READING
Outbound RPC batches in the capture	512	All state traffic; no media traffic crosses this path.
Distinct <code>ofs</code> values vs batches	260 / 512	≈2× duplication. Every state fact is posted twice because two sessions are bound and the bridge fans out per session.
Descents in the <code>ofs</code> sequence	106	Two independent counters interleaved, lapping each other. From outside, the session's ordering guarantee is unreadable — a consumer cannot sort these posts into one timeline.
Median gap between posts	0.06 s , max 8.56 s	Bursty: three reports within ~50 ms, then silence for 2 s. The phone is asked to absorb a burst and then extrapolate position on its own.
Report families used	4 of 10 that official clients emit	No ad state, no up-next, no subtitles, no speed, no autoplay-mode: the phone keeps defaults for six dimensions of the session.
<code>cpn</code> in every report	constant	The phone cannot separate playback instances. See §4.2.

Fig. 2 The phone renders a mirror of the receiver's reports; the add-on feeds 4 of the 10 event families

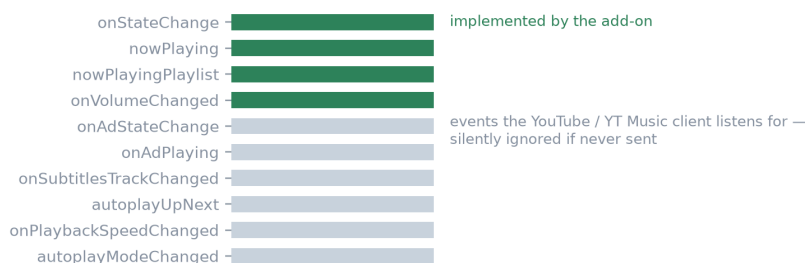


Fig. 8 The phone's model is fed by ten event families; the receiver feeds four. The six silent ones are not failures — they are the phone rendering defaults it cannot correct.

Conclusion from the numbers: the receiver is not under-reporting state out of laziness — it is reporting the *right* facts in a shape that cannot be reconciled: duplicated across channels, with unreadable ordering, no session identity, and gaps in the vocabulary the phone expects. That is a structural problem, and it is fixable structurally.

Why there are so many sync bugs: five structural causes

None of the recurring bugs is a typo: each is the predictable consequence of an architectural choice made in §3–§6. They collapse into five buckets, three of which the redesign in §9 removes outright rather than fighting them bug-by-bug.

C1 · Identity is carried, not owned

Cause: reports are interpretable only against the `listId` + `currentIndex` the phone last sent for that queue; the receiver keeps them in RAM and re-derives the index after every edit.

Symptom: the TV advances, the phone stays frozen on the previous item (bar 0:00, restart glyph). Restart mid-queue re-announces a queue the phone cannot match. A wrong index silently mis-syncs the whole queue.

Retired by: a persisted session record keyed by `listId`, plus an index that is only ever derived from the stored queue (§9 R2, R5).

C2 · No acknowledgement, no idempotency

Cause: commands are fire-and-forget; a batch can be dropped (relay non-determinism), relayed late, or duplicated (burst re-delivery). Frame `code` dedupe only helps if the duplicate arrives in order.

Symptom: a play/pause/seek that never arrives (receiver still in the previous state, phone already showing the new one); a duplicate `setPlaylist` triggering two resolves; commands delivered one event late.

Retired by: dedupe on a command identity (`listId+videoId+offset`) and a reconcile pass that re-asserts the intended state instead of assuming delivery (§9 R3, R7).

C3 · The same fact has four homes

Cause: the queue, its order and the current item exist simultaneously in the phone's model, the bridge's dict, Kodi's own playlist, and the info label Kodi exposes. Kodi advances its playlist by itself; the bridge learns only by polling.

Symptom: queue drift after a local pick; auto-advance not reflected upstream; titles that no longer match their items; index re-derivation that fights the phone's copy.

Retired by: exactly one in-process owner of the queue; Kodi's playlist becomes a projection, never a source (§9 R1, R4).

C4 · Reality is inferred, twice a second, forever

Cause: on Kodi 21 the service process receives no player callbacks at all, so pause/resume/track-change/lane are guessed from clock stalls, an info label and window repairs.

Symptom: pause flapping; a stall long enough to look like a pause; a track change missed entirely so reports keep describing the old item; visualiser/window thrash between the music and video lanes.

Retired by: one signal source per fact with an explicit confidence flag, and a reconciliation step that treats "unknown" as a state rather than guessing (§9 R6, R8).

C5 · Fan-out duplicates the model instead of the message

Cause: one receiver, two Lounge channels (YouTube + YT Music), each with its own listener, poster, counter and queue — and a report loop that pushes every fact to every session.

Symptom: doubled state traffic; interleaved offsets that destroy ordering (Fig. 7b); coalescing rules with exceptions; a second channel whose stale copy can win.

Retired by: one snapshot, N subscribers, one publisher per channel — duplication moves to the transport where it costs nothing (§9 R9).

Fig. 9 The five causes. C1, C3 and C5 are architectural and disappear with the redesign; C2 and C4 are protocol/physics and can only be bounded.

8.1 Symptom → cause lookup

WHAT THE USER SEES	CLASS	FIRST THING TO CHECK
TV advances to the next item; the phone still shows the previous one, bar at 0:00, restart glyph.	C1	Did the report echo the stored <code>listId</code> and an index re-derived against the stored queue?
Play/pause/seek does nothing once, then works after a second tap.	C2	Was the command delivered at all (receiver log), or dropped by the relay?
Two resolves / double playback start around a cast.	C2	Duplicate relay burst — is the command handler idempotent?
Queue order in the phone disagrees with what plays next.	C3	Which copy won: bridge dict, Kodi playlist, or title cache?
Phone shows play while the TV is paused (or the reverse).	C4	Was the transition inferred from a stall heuristic, and did its report go out?
Phone progress bar stops updating but playback continues.	C4	Is the tick still publishing, and is it gated on a state flag that is stale?
The device appears twice in the cast list, or state from one channel wins at random.	C5	How many sessions are bound, and does each publish independently?
Casting hangs right after the device shows up in the picker.	—	Not a state bug: <code>screenId</code> mismatch between discovery and the bound listener, or a port squatter.

The pattern behind them

Each cause is a **gap between the model the phone holds and the model the receiver holds**, closed after the fact by another heuristic, another report or another repair loop. Every fix that follows that pattern makes the system larger and adds a new way to be wrong — the observed trajectory exactly: 1561 lines in the bridge, four writers, three report families per 2 s tick per session, two channels, and a growing list of special cases (watchdogs, hysteresis, coalescing exceptions, window repair). Reliability does not come from more state being sent; it comes from **less state existing in more than one place**.

A smaller architecture: one model, one writer, N subscribers

The protocol cannot be changed — the phone is Google's. What *can* be changed is everything between the frame parser and the reports: the receiver need not keep four copies of the queue, guess its own state twice a second, or duplicate its model per channel.



Fig. 10 Target shape: many event sources, one state owner, one publisher per channel, and a player projection that is derived rather than policed.

9.1 The rules that do the work

#	RULE	WHY IT KILLS BUGS
R1	One in-process owner of session state; nothing else holds a copy.	Removes C3 entirely. "Which copy is right?" stops being a question that can be asked.
R2	Persist the session record — <code>listId</code> , playlist, index, item, position, <code>cpn</code> — not just the pairing tokens.	Restart no longer destroys the identity the phone indexes its queue by. Removes the most confusing class of C1 failures.
R3	Every state change is a <i>reduction</i> of an event; there is no direct field write.	A state transition happens in one place, so it can be logged, tested, replayed and reasoned about. Four writers become one.
R4	Kodi's player and playlist are <i>projections</i> of the snapshot, rebuilt on change — never polled for truth, never repaired on a timer.	Removes the drift/repair class of bugs and most of the bridge's line count. Window/lane repair loops disappear.
R5	<code>currentIndex</code> is derived from the stored queue at the moment of publication, never carried.	Removes the second identity mismatch (index vs list) without asking anything of the phone.
R6	Signals carry confidence: each fact has one source and an <code>unknown</code> value that is legal to publish.	Publishing "unknown" is recoverable; publishing a guess is not. Kills the heuristic stakes in C4.
R7	The publisher coalesces by <i>snapshot version</i> , not by RPC tag, with no exceptions.	A backlog can only ever collapse to the newest truth. Removes the hand-carved <code>nowPlaying</code> exception and stale-state ordering bugs.
R8	Reconcile on a 1 s tick: compare the snapshot against the player, and route any disagreement back through the reducer as an event.	Turns the polling loop from a state <i>writer</i> into a state <i>checker</i> . Latency stays; wrongness goes.
R9	One model, N subscribers: a channel is a transport detail, never a second state.	Removes C5. Adding a channel adds a socket and an offset — no queue, no listener logic, no fan-out loop.
R10	Report vocabulary is a declared set, and gaps are visible in the log.	What the phone is missing becomes measurable (Fig. 8) instead of being discovered as a UI mystery.

From one snapshot to many batches

The reducer is the only place truth changes. The publisher is the only place truth leaves the process. Between them sits a contract simple enough to hold in one's head — which is the point, because the current design's failure mode is that no single person can state what the receiver believes and why.

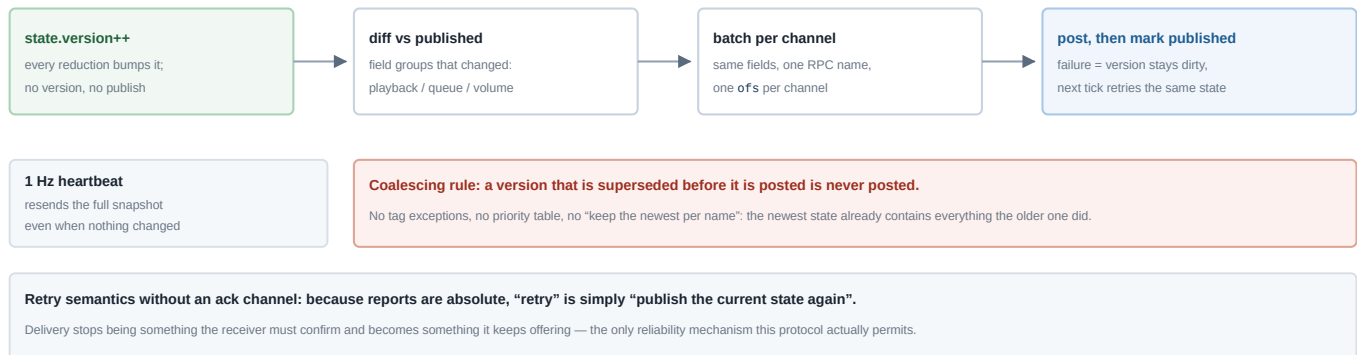


Fig. 11 The publish path. One rule replaces the per-tag coalescing, its `nowPlaying` exception, and the position loop's direct writes.

SHAPE OF THE REDUCER

```

def reduce(state, event):          # the ONLY writer
    if event.kind == "setPlaylist": # identity arrives once, whole
        return state.with_queue(event.list_id, event.video_ids,
                                index=index_of(event.video_ids, event.video_id),
                                position=event.current_time, cpn=new_cpn())
    if event.kind == "pause":       return state.with_play(PAUSED, source="phone")
    if event.kind == "kodi_advanced": # Kodi moved on its own – adopt it
        return state.with_item(event.video_id, source="player")
    if event.kind == "signal_unknown": return state.with_play(UNKNOWN)
    return state                   # unknown events are inert, never fatal
  
```

Idempotence falls out of this shape: replaying the same event twice produces the same state, which is what makes a duplicated relay burst harmless rather than a double resolve.

9.3 Before and after, as it affects the wire

TODAY		AFTER	
Writers of state	4 concurrent paths (dispatch, position loop, resolver, repair loops)	Writers of state	1 reducer; everything else emits events
Reports per change	3 families × N bound sessions	Reports per change	1 batch per channel that has changed
Ordering	unreadable across channels (106 counter descents measured)	Ordering	one counter per channel, monotonic by construction
Backlog behaviour	per-tag coalescing with a hand-carved exception	Backlog behaviour	superseded versions are never posted; newest state wins by definition
Identity across restart	lost (<code>listId</code> in RAM only)	Identity across restart	persisted with the session record
Player truth	re-inferred from clocks and labels twice a second	Player truth	reconciled once per tick, disagreements routed back as events

Getting there without a rewrite

#	STEP	RETIRES	EFFORT	NOTE
1	Extract <code>SessionState</code> from the bridge: a plain data class plus the reducer, no Kodi imports.	—	small	Pure refactor. Unit-testable offline, and the emulator can assert on snapshots instead of report strings.
2	Persist the session record (listId, playlist, index, item, position, cpn).	C1	small	The existing settings blob already holds tokens; this is one more key. Highest ratio of bugs fixed to lines written.
3	Route every mutation through the reducer; delete direct field writes.	C3	medium	The position loop stops writing state and starts emitting events.
4	Replace the report loop with a version-checking publisher; drop the per-tag coalescing and its exception.	C5, ordering	small	One publisher instance per channel, one snapshot argument.
5	Make the Kodi playlist/windows a projection; delete the timer-driven repair loops.	C3, C4	medium	Largest deletion in the codebase; verify with the existing lane/window scenarios before removing anything.
6	Add command dedupe on (listId, videoId, offset) and publish <code>UNKNOWN</code> for un-believable signals.	C2, C4	medium	Dup-relay bursts become harmless; pause flapping becomes “unknown” rather than a wrong claim.
7	Declare the report vocabulary as a table with coverage logging; add the missing families where the phone needs them.	UI staleness	small per family	Ad state and up-next are the highest-value additions; speed and subtitles are cosmetic-first.
8	Add a corpus-replay test: feed the capture through the reducer and assert the <i>final snapshot</i> , not individual reports.	regressions	small	The emulator already replays the capture — the assertion should move up a level.

Expected effect

- **Bug classes removed outright:** C1, C3, C5 — identity loss, multi-home facts, duplicated model.
- **Bug classes bounded:** C2 (dedupe + reconcile re-asserts intent instead of assuming delivery), C4 (latency remains, wrongness does not).
- **Wire traffic:** roughly halved by publishing one snapshot per channel instead of one report per fact per channel — and the ordering becomes readable again because each channel has one counter.
- **Debuggability:** one log line per state version answers “what did we believe, and why?” — which is currently impossible across four writers and two counters.

What not to do

- **Do not add reports to fix a phone that ignores reports.** If the phone discards a state it cannot map, more state changes nothing — the decodability problem is identity (§4.2), not volume.
- **Do not add polling.** The 2 s tick is already a guess; a second poller makes two guesses that disagree.
- **Do not paper over a disagreement by storing another copy** — that is precisely how C3 grew.
- **Do not treat HTTP 200 as delivery.** Verify against the phone's rendering, or against a snapshot assertion in the emulator.

The one-sentence version

The current design sends *everything* the phone might want, repeatedly, from several places, and hopes the phone assembles a coherent picture — the fix is to publish *one* authoritative, versioned picture per channel and let the phone do what it already does well: render the latest snapshot it can map.

Protocol and evidence reference

A · Endpoints

B · Screen bind parameters

C · Playback state codes

CALL	PUF	PARAM	VALUE / MEANING	CODE	MEANING	TRAP
pairing/generate_screen_id	Screen id (play)	device id	LOUNGE_SCREEN (receiver) REMOTE_CONTROL (sender)	1	Stopped / unstopped	—
pairing/get_lounge_token_batch	Durable channel credentials (~14)	loungeIdToken	screenId — the channel key the screen's channel credential	0	Ended	Reporting 0 while loading shows length 0 + restart button
pairing/get_pairing_code	12-codes shown to the user	capabilities	dsp, mic, dpa, ntb, que, music : que / music let YT Music target the device	2 3	Playing Paused Buffering / starting	—
pairing/get_screen	Phone resource code → screenId + name	RID / AID / SID / resourceid	request id, app id, session id, channel id	5	Video cued	—
bc/bind		Everything else: handshake, long-poll, commands, reports				

D · Evidence index

SOURCE	USED FOR
kodi-yt-caster/resources/lib/lounge/{session, listener}.py	\$1, \$5, \$6 — frames, ofs, RPC names, frame-code dedupe
kodi-yt-caster/resources/lib/player_bridge.py	\$6, \$8 — emission cadence, polling heuristics, the four state writers
kodi-yt-caster/emulator/**	\$6 — wire fidelity, duplicate relay bursts, assertions on receiver reports
protolab/README.md	\$1, \$5 — endpoints, framing, JSON-not-protobuf, handshake shape
pyytlounge (sender-side reference implementation)	\$4 — event vocabulary, command shapes, session parameters
lounge-capture/tv_full.jsonl (2026-09-14)	\$7 — 512 posts re-measured: duplication, offset interleaving, cadence

Compiled from the receiver's own source, its protocol notes and the 2026-09-14 device capture. Statements about Google's implementation describe observed client behaviour, not published specification — Lounge is undocumented, and every field quoted here should be re-verified against a fresh capture before being relied upon.